

EvoRec

智能商品推荐平台 技术栈选型报告

V1.0 · 个人研究与可展示系统 · 2026 年 9 月 14 日

从推荐研究的需要出发，说明算法、数据、检索、服务与展示技术的选择依据，明确替代方案、实施边界及验证条件。

算法可研究

复现强基线
保留实验可修改性

结果可验证

固定数据与版本
衡量质量与真实成本

产品可展示

完成真实交互流程
保持个人可维护

阅读导航

- 01 摘要与决策依据
- 02 总体架构与职责划分
- 03 算法主体：Python 与 PyTorch
- 04 商品表示与检索：Sentence Transformers、Faiss
- 05 离线数据：Parquet 与 DuckDB
- 06 服务与任务：FastAPI、Pydantic
- 07 业务状态：PostgreSQL 与版本发布
- 08 展示层：React、TypeScript 与图表组件
- 09 实验管理：配置、曲线与结果文件
- 10 性能扩展：C++20、CMake 与 pybind11
- 11 环境、部署与资源成本
- 12 替代方案与重新评估条件
- 13 实施顺序与验收结论
- 14 参考资料与证据说明

配套 EvoRec 产品需求文档。本文为选型与实施建议；系统实现、环境兼容性及性能收益仍需验证。

01 摘要与决策依据

EvoRec 面向持续更新的商品库，研究如何结合用户历史、商品内容与生成式检索，在给定计算预算下提高推荐质量，并让新商品及时参与推荐。最终作品需要同时具备可复现的算法实验、完整的商品发布流程和能够现场操作的展示页面。

本报告建议采用 Python 与 PyTorch 作为算法主体，使用 Sentence Transformers 构建商品内容表示，以 Faiss 完成向量检索；离线数据采用 Parquet 与 DuckDB，在线业务数据采用 PostgreSQL；服务层采用 FastAPI 与 Pydantic，前端采用 React、TypeScript、Vite、Ant Design 与 ECharts；实验配置和记录采用 Hydra、TensorBoard 与结构化结果文件。C++20、CMake 和 pybind11 作为有条件引入的性能扩展，部署采用 Linux 与 Docker Compose。

这一选择围绕三个目标展开：降低算法迭代与基线复现的成本；使数据、模型和实验结果能够追溯；将研究能力转化为可靠的演示流程。技术栈的规模控制在个人能够维护的范围，研究深度通过问题定义、实验设计和经过验证的改进体现。

评价标准

评价维度	本项目关注的问题	对选型的影响
算法适配	能否修改模型、损失、采样和检索策略	优先兼容研究实现的 Python 与 PyTorch
结果可信	不同策略是否使用相同输入与指标口径	固定数据协议、配置、版本和评估入口
个人维护	一个人能否定位训练、接口和展示故障	单机起步，按职责拆分模块与进程
展示完整	能否实际推荐、反馈、导入新品和比较策略	提供业务接口、持久状态与交互页面
性能可测	能否区分模型、检索、排队与数据传输开销	先分段测量，再选择优化工具
演进成本	更换模型、索引或服务方式会影响哪些模块	用明确的数据契约隔离变化

本报告采用定性比较，不给框架设置缺少实验依据的分数。推荐结论是面向 EvoRec 需求的工程判断，不代表同类技术的通用优劣排名。

02 总体架构与职责划分

技术清单

所属层	建议技术	主要职责
算法与训练	Python、PyTorch；按基线需要使用 Transformers	用户序列建模、生成式推荐、路由学习与训练评估
商品表示	Sentence Transformers	标题、类别与描述的向量化
离线数据	Parquet、DuckDB；NumPy 辅助张量准备	数据快照、时间切分、统计与训练输入
候选检索	Faiss	精确或近似向量检索、索引保存与加载
业务服务	FastAPI、Pydantic、Uvicorn	接口契约、请求校验与服务运行
状态持久化	PostgreSQL；SQLAlchemy 管理数据访问	会话、反馈、批次、发布状态与运行记录
展示界面	React、TypeScript、Vite	推荐体验、商品管理与策略对比
组件与图表	Ant Design、ECharts	业务组件、指标图表与失败样例查看
实验管理	Hydra、TensorBoard、JSON 结果文件	配置组合、训练观察与结果归档
性能扩展	C++20、CMake、pybind11	经测量确认值得迁移的局部计算
运行与交付	Linux、Docker Compose、Git	环境固定、服务启动与代码追溯

两条工作链路



推理进程加载经验证的产物；训练与演示按资源预算调度。

离线链路为“数据快照 → 特征与样本 → 模型训练 → 统一评估 → 版本化产物”。在线链路为“页面请求 → 接口校验 → 历史与商品状态 → 受控推理 → 排序过滤 → 返回结果”。离线任务产出的模型、商品映射和索引，经验证后供推理进程加载。

训练任务、商品构建任务与在线推理承担不同工作。首版采用一个推理进程管理目标模型和检索索引，并限制并发及等待队列；后台构建任务按记录执行，进度可查询。部署期间让训练与正式演示错峰运行，避免单块 GPU 上的资源竞争破坏延迟测量。

在代码层面，数据处理、算法、评估、服务和前端按模块组织。在运行层面，只有确需隔离资源或生命周期的部分拆成独立进程。跨进程使用明确的请求和结果结构，版本化文件承载较大的模型与索引数据。

03 算法主体：Python 与 PyTorch

选择原因

EvoRec 的主要开发活动包括阅读基线、调整序列输入、修改损失函数、控制候选生成方式和组织消融实验。Python 适合连接这些步骤，PyTorch 用于张量计算、模型训练与推理。保持同一套模型定义，有助于减少训练实现与展示实现之间的偏差。

项目的直接参考之一 LIGER 提供了训练、语义编码和实验配置相关代码，其环境文件包含 Python、Transformers、Sentence Transformers 与 Hydra。采用相近的算法环境，可以从官方实现建立可工作的起点。LIGER 官方实现、官方环境文件

本项目据此选择 PyTorch，主要收益是降低复现与修改的衔接成本。例如，研究路由决策时，可以复用已有用户表示，调整策略损失，并在同一评估入口比较纯生成式、内容检索和混合方案。每项改动都能关联到具体配置与结果。

与其他路线的取舍

全 C++ 训练路线会增加基线移植、训练工具接入和模型维护的工作量。对于以推荐算法研究为主要目标的个人项目，这部分工作应在具有明确收益时投入。C++ 保留在性能扩展位置，Python 先完成可验证的算法实现。

TensorFlow 或 JAX 可以作为其他研究项目的选择。本项目已经选定的参考实现和复现路径更贴近 PyTorch，框架迁移本身不会为研究假设增加证据，因此首版不安排迁移。

模型封装和训练框架也应按需引入。只有基线依赖 Transformers 时才保留对应组件；自定义路由与评价逻辑保持可读，避免把关键实验步骤封装到难以检查的位置。

使用边界与验收

先验证小样本上的训练、保存、重新加载与推理，再扩大数据规模。正确性检查覆盖历史截断、填充掩码、物品映射、损失输入和评估候选集合。模型推理使用正确的评估模式，关闭不需要的梯度记录。

算法验收要求完整记录训练配置和基线改动，报告整体、模型冷物品与低频物品分组结果。选择 PyTorch 并不保证算法优于基线；只有统一协议下的对照实验能够支持效果结论。

04 商品表示与检索：Sentence Transformers、Faiss

内容表示为何独立建设

新商品可能在当前商品库中有效，却没有出现在训练交互中。只依赖训练阶段的商品 ID 表示，不能充分处理这一场景。EvoRec 因此为标题、类别和描述建立内容向量，使新品发布后具备参与内容检索的条件。

Sentence Transformers 提供文本编码接口，可用于将文本转换为向量；不同模型还可能对查询和文档采用不同的输入约定。Sentence Transformers 使用文档

首版选择与复现方案一致、能够本地加载的文本编码器，并固定权重版本、字段拼接规则、截断长度和归一化方式。商品编码批量计算并保存，在线请求复用已有表示。具体模型名称和向量维度在小规模实验后锁定，不将任意预训练模型视为天然适配商品推荐。

用户表示与商品表示必须处于可比较的空间。用户向量如果来自独立序列模型，需要通过训练或明确的映射对齐；维度相同并不意味着可以直接计算有意义的相似度。生成式语义 ID 的构建与文本向量编码也分别管理。

Faiss 的选型依据

Faiss 提供不同的向量索引。IndexFlatIP 执行精确内积搜索；当向量事先归一化时，可用于余弦相似度检索。HNSW、IVF 等索引用于不同的近似检索取舍。Faiss 索引说明

首版先使用精确检索建立参照。这样，当推荐效果不佳时，可以先排查表示与模型，不必同时猜测近似搜索是否漏掉了候选。商品规模增长、精确检索出现实测瓶颈后，再以精确结果为参照评价近似索引的召回损失、延迟和内存。

对于本项目，Faiss 的价值在于检索能力直接嵌入推理进程，方便单机测量和索引版本管理。独立向量数据库在多客户端共享、远程访问和运维需求增加时可再评估。Faiss 本身不承担商品事务、权限、下架状态或整个系统的发布一致性。

动态商品库的约束

商品内容向量可加入候选库，不代表生成模型一定能生成其语义路径，更不代表新品一定进入 Top 10。系统需要分别验证商品有效性、内容检索可达性、语义编码合法性和实际排序位置。

索引内部编号与稳定商品 ID 必须有明确映射。首版通过后台构建不可变索引版本并切换读快照，减少并发修改索引的复杂度；下架名单作为当前业务状态持续生效。验证覆盖编号错位、重复候选、候选不足与版本切换。

05 离线数据：Parquet 与 DuckDB

为什么分离离线分析与在线业务

离线实验需要重复扫描交互记录、按时间筛选、统计商品频率并生成训练样本。在线服务需要快速读取会话、写入反馈并处理发布状态。两类任务的数据访问方式不同，应各自拥有清晰的数据入口。

Parquet 用作离线快照格式，DuckDB 用于查询与聚合。DuckDB 可以直接读取和写出 Parquet，并支持在读取时利用列选择与过滤条件减少不必要的数据处理。[DuckDB Parquet 文档](#)

项目因此将清洗后的交互和商品信息按版本保存，通过 SQL 生成统一的数据切分与分组统计。Pandas 可用于小规模探索，NumPy 可用于准备紧凑的训练数组。训练时按经过测量的批量读取方式加载，避免每个样本都重新执行一次分析查询。

对实验可信度的贡献

每个训练、验证与测试快照记录时间边界、去重规则、可用商品规则和文件校验值。所有基线使用相同评估事件集合；不同方法确需不同处理时，记录差异及其影响。数据工具负责执行协议，协议本身仍须由项目定义并检查。

用户历史只能包含请求时刻之前的行为。词表、频率统计、归一化参数和可学习编码器的拟合范围也需与协议一致。测试集用于最终报告，调参和阈值选择使用验证集。

现有 PRD 使用 Amazon Reviews 2023。历史商品资料未必对应当时的真实内容，首次观测到交互的时间也不等同于真实上架时间。报告需要标记这些限制，将离线代理场景与平台记录的新品导入场景分别分析。

与替代方案的取舍

全部使用 CSV 便于查看，但随着数据快照增多，需要额外管理类型、读取和字段约定。全部放入业务数据库则会把重复分析负载与展示服务混在一起。Parquet 与 DuckDB 为本项目提供独立、可复制的离线数据路径。

Spark 等分布式处理方案在数据量和单机资源形成实测约束时再评估。首版先测量扫描时间、峰值内存和训练输入吞吐，确定单机方案的适用范围，不承诺处理未经验证的数据规模。

06 服务与任务：FastAPI、Pydantic

选择原因

服务层需要承接推荐、反馈、导入、发布与对比请求，并使前端能够检查参数和错误。FastAPI 基于类型声明和 Pydantic 组织数据校验，支持 OpenAPI 描述与交互式接口文档。[FastAPI 功能文档](#)

这些能力适合将研究代码整理为明确的产品接口。例如，推荐请求携带会话和策略，响应包含商品列表、实际执行路径与版本；导入请求在进入任务流程前完成字段校验；失败使用稳定的错误结构返回。

项目采用 FastAPI，是为了使 Python 算法与接口契约容易衔接。若采用 Flask，需要另行选定校验与接口文档的组织方式；若采用 Django，则应确认完整业务体系是否符合项目范围。当前重点在推荐、商品生命周期与实验展示，故选择显式组合所需能力的服务结构。

异步接口与模型执行分别设计

异步适合在等待网络或数据库等操作时协调请求，CPU 与 GPU 计算仍需要相应的执行资源。FastAPI 文档明确区分并发与并行。[FastAPI 并发与并行说明](#)

本项目建议由专用推理进程加载模型，通过有限队列控制等待和执行数量。接口设置超时和过载处理，记录排队、模型、检索与后处理耗时。增加接口进程时，不随之重复加载 GPU 模型。请求超时后，还需处理已排队任务的取消或失效，避免后台无效工作持续积压。

对于商品构建与批量评估，任务先写入 PostgreSQL，再由后台进程领取执行。首版采用单个后台执行者，以任务 ID 保证重试可辨认；进程中断后的任务需要能够恢复或明确标记失败。仅把函数放入接口进程的后台回调，不能满足持久任务要求。

验收关注点

同一反馈事件的重复提交不重复计数；相同批次和内容重试返回既有任务；模型异常可触发有记录的回退；服务繁忙时有明确响应。接口可以正常返回，不等于推荐结果正确，因此请求校验、算法结果和业务状态需要分别验证。

07 业务状态：PostgreSQL 与版本发布

为什么选择关系数据库

商品、导入批次、演示会话、反馈事件和发布版本之间存在明确关系，并需要唯一性约束及状态转换。

PostgreSQL 提供事务，用于将多步数据库修改作为一个整体提交或回滚。PostgreSQL 事务文档

例如，反馈事件保存与会话版本更新应协调完成；导入批次区分待处理、处理中、待发布、已发布与失败；同一商品 ID 不应无意创建多份记录。SQLAlchemy 用于集中管理数据访问，数据库约束仍作为最终检查。

SQLite 可以支撑轻量本地原型。最终展示版选择 PostgreSQL，是考虑到接口、后台任务与管理页面会同时访问共享状态，并希望开发与部署沿用同一种数据库。这是本项目的维护选择，不意味着 SQLite 无法提供事务或无法承载演示。

MySQL 也是可行替代，本项目不依赖 PostgreSQL 独有功能。若实施者已经熟悉 MySQL，可以保留相同的数据约束和事务设计后替换。选定 PostgreSQL 的目的在于统一实现，不将数据库品牌作为推荐效果或性能优势的证据。

数据库事务不能替代模型发布协议

模型权重、物品映射、语义编码、向量索引通常保存为文件。数据库事务无法自动将这些文件的生成与推理进程内存切换合并为一次原子操作。因此，发布必须单独设计协议。

本报告建议采用以下顺序：写入新的不可变产物目录；验证文件校验值、映射数量、维度和模型兼容性；由推理进程预加载并完成健康检查；进入发布屏障；提交目标活动版本并切换服务快照；确认可用后恢复接收新请求。

首版通过单个推理进程和短暂的发布屏障明确切换边界：已接收的请求持有旧快照完成，新请求在切换确认后使用新快照。若切换过程中崩溃，服务在恢复并对齐活动版本之前保持未就绪状态。未来多实例部署需要重新设计版本确认和流量切换。

下架状态独立于历史模型包。处理下架时协调状态更新和请求接收，使下架确认后发起的新请求使用最新过滤条件；回滚模型或索引时保留当前下架名单。版本回滚的测试必须检查商品是否被错误恢复推荐。

08 展示层：React、TypeScript 与图表组件

页面选择服务于完整演示

EvoRec 需要展示推荐卡片、商品导入状态、策略差异和实验结果，页面具有持续交互。React 用于组织组件与状态，TypeScript 用于约束前端数据结构，Vite 用于开发与构建。React 官方文档提供了使用 Vite 等构建工具建立应用的路径。[React 应用构建文档](#)

本项目没有首版搜索引擎收录或服务端渲染需求，因此选择浏览器单页应用配合独立接口。前端按功能组织，并处理加载、失败、重试、空数据和会话重置，避免将展示能力限于一组静态截图。

Vue 同样可以完成这些页面。选择 React 是为了固定组件和工具组合，减少实现时反复切换方案。若已有成熟的 Vue 能力，替换前端不会改变算法研究目标。Streamlit 或 Gradio 可用于早期模型检查，正式展示页面则按 PRD 的状态流转与交互验收。

Ant Design 与 ECharts 的分工

Ant Design 提供 React 组件与 TypeScript 支持，可用于表单、表格和状态展示；ECharts 提供图表与交互能力。[Ant Design 文档](#)、[ECharts 官方介绍](#)

采用这些组件的目的，是把界面工作集中在业务行为与证据呈现。策略对比需要固定用户历史和商品库，展示不同 Top 10；效果看板呈现 NDCG、冷物品分组与延迟曲线；商品工作台展示失败原因和发布进度。

指标由经过版本记录的后端评估结果提供，前端负责展示和筛选。缺失值显示为“暂无数据”，样本数、数据来源和模型版本与图表一起出现。演示点击不换算成真实商业收益，离线提升也不作为线上 CTR 提升展示。

验收要求

用户切换后状态隔离；重复点击不会造成重复事件；发布失败不会显示成功；刷新能够恢复持久任务状态。前端还需区分“已进入候选库”和“已获得推荐曝光”，使演示者能够准确解释新品处理结果。

09 实验管理：配置、曲线与结果文件

为什么选择轻量组合

推荐研究需要比较序列长度、负采样、候选数量、生成参数和路由阈值。Hydra 支持分层配置组合与覆盖，适合将这些变化组织成可检查的实验配置。[Hydra 官方文档](#)

TensorBoard 用于观察训练标量和变化趋势，PyTorch 提供对应的日志写入接口。[PyTorch 日志接口](#)

本项目建议 Hydra 管理实验输入，TensorBoard 辅助训练诊断，JSON 或 Parquet 结果文件保存指标及逐请求输出。最终展示看板读取归档结果。TensorBoard 不承担模型注册、数据版本管理或指标口径定义，这些信息由项目的运行清单保存。

每次运行记录代码提交、未提交改动、完整配置、随机种子、依赖版本、数据校验值、设备信息、模型与索引标识。保留失败运行及错误原因，避免只呈现效果最好的记录。

可复现不等于任意环境逐位相同

PyTorch 官方说明指出，跨版本、平台或设备不保证完全一致的结果，即使设置相同随机种子也存在边界。

[PyTorch 可复现性说明](#)

项目因此固定环境和协议，并区分散逻辑一致性与数值容差。主要结论采用多个独立训练种子，报告均值与波动；正式关键比较建议至少三次独立训练，在预算允许时增加重复。路由阈值在验证集上确定，最终测试集不用于挑选最佳配置。

成本比较必须覆盖真实执行

路由器只使用请求时可获取的特征，不能访问目标商品或未来行为。若路由特征需要先执行昂贵路径，其计算必须计入总成本。不能先完成所有路径再选择结果，却只计算被选路径耗时。

统一记录端到端耗时、分段耗时、回退率、候选数量与资源使用。批量策略对比作为独立任务执行，其多路径成本不冒充日常单次推荐成本。W&B 或 MLflow 在需要远程协作、共享运行记录或模型注册时再评估；复现代码已有的记录接口通过适配保留所需数据。

10 性能扩展：C++20、CMake 与 pybind11

C++ 的投入条件

EvoRec 保留 C++ 扩展能力，以发挥实现者的语言基础，但将投入建立在性能测量上。推荐请求可能主要耗时于 GPU 模型、CPU 候选处理、索引检索或队列等待。只有定位到可改进的部分，才能决定是否值得迁移。

候选模块包括语义编码前缀查询、批量去重与合法性过滤、局部 Top-K 处理，以及需要严格生命周期管理的索引快照。每个模块先有可验证的参考实现，再考虑数据布局、批量接口和编译优化。

CMake 用于固定构建选项和组织扩展模块，pybind11 用于连接 Python 与 C++。pybind11 文档说明，从 Python 进入 C++ 时不会隐式释放 GIL；释放与重新获取必须遵守 Python 对象访问要求。pybind11 并发与 GIL 说明

为什么采用局部扩展

保留 Python 模型和评估入口，只迁移边界清晰的计算，可以让算法迭代与性能工作并行演进。调用应尽量批量化，明确数组类型、连续性、数据所有权和异常行为。若每个候选都跨语言调用一次，接口开销可能抵消计算收益。

对于现有 Faiss 或张量算子已经高效完成的工作，需要先与库实现比较。调用底层使用 C++ 的库属于集成工作；项目自研贡献应准确对应自己实现并验证的模块。

如何证明优化有效

正确性验证覆盖空输入、重复商品、全部被过滤、分数相同、数量不足和版本切换。整数 ID、合法性与去重结果应一致；涉及浮点运算时明确误差容限及并列分数规则。

性能比较使用相同硬件、输入、线程数和负载，先预热，再以平衡顺序重复运行，报告模块耗时与端到端 p50、p95。GPU 测量需要正确同步或使用相应计时机制。收益不足时保留简单实现，同时记录结论；不为了体现 C++ 而维护没有实际收益的双份代码。

11 环境、部署与资源成本

环境选择

目标运行环境为 Linux。本地可在 Windows 编辑代码，将训练和最终验证放在固定的 Linux 环境；具体使用本机、WSL2 或远程主机，依据可用设备确定。GPU 型号、显存和驱动尚未确认，因此本报告不指定显存容量承诺或整套可直接安装的版本组合。

基线复现环境与展示服务环境分别锁定。LIGER 官方环境当前指定 Python 3.10、Sentence Transformers 2.7.0、Transformers 4.39.3 等依赖，应结合仓库提交与安装说明使用，而不是自动升级全部组件。LIGER 环境文件

可供部署的版本需要通过训练、小样本推理、权重加载和接口验证；不能因为复现环境可以运行，就直接认定适合长期公开服务。模型产物在两个环境间建立兼容性检查，前端保存依赖锁文件，C++ 记录编译器和编译选项。

Docker Compose 的作用与边界

Docker Compose 使用配置文件描述服务、网络和数据卷，并统一管理多个容器的启动。Docker Compose 文档

本项目利用它组织前端、API、推理与任务执行服务和 PostgreSQL。模型与数据库使用持久目录或数据卷，启动时先完成连接检查和模型预热，再进入就绪状态。发布前验证能够在新目录恢复示例数据和指定模型版本。

Compose 提高启动过程的一致性，但不保证 GPU 驱动兼容，也不自动提供高可用或无停机发布。首版以单机可重复演示为验收范围，版本切换允许按已定义的屏障短暂暂停接新请求。

资源预算的计算方法

纯 FP32 向量存储约为“商品数 × 向量维度 × 4 字节”。例如 10 万个 384 维向量约为 153.6 MB，100 万个约为 1.536 GB，均采用十进制单位。这是算术估算，未包括索引结构、模型、元数据、查询缓冲以及新旧版本同时驻留的内存。

训练时间与显存还受到序列长度、批量大小、优化器、模型规模和精度的影响。先完成一个小规模完整实验，记录峰值显存与每轮耗时，再规划后续实验量。预算路由的价值最终也必须用质量与真实成本的关系验证。

12 替代方案与重新评估条件

下表记录针对本项目的取舍，不构成相关技术的性能排名。出现右侧条件时，应重新测量和评审，而不是按技术清单机械增加组件。

当前选择	可评估的替代或扩展	重新评估条件
Python 与 PyTorch 主体	C++ 模型服务、其他训练框架	目标部署受限，或另一关键基线要求不同环境
文本内容表示	图文多模态表示	有合适图像数据，且分组实验显示文本信息不足
Faiss 进程内检索	独立向量数据库	多服务共享、远程索引管理成为实际需求
Parquet 与 DuckDB	分布式数据处理	单机处理时间或内存超出已确定预算
PostgreSQL	MySQL; SQLite 原型	已有维护经验或部署限制显著改变成本
数据库任务表与单执行者	专用任务队列	出现多个执行者、复杂调度或明确吞吐瓶颈
直接读取业务状态	Redis 等缓存	测量证明数据库访问占主导，且已定义失效规则
React 单页应用	Vue、服务端渲染框架	已有前端积累，或增加公开收录与渲染需求
本地实验记录	W&B、MLflow	需要共享运行管理、协作或模型注册能力
单机 Compose	多实例及集群部署	有持续流量、可用性要求与对应维护资源

13 实施顺序与验收结论

按交付结果推进

阶段	实施重点	完成证据
A 算法基础	数据协议、内容表示、强基线与统一评估	小规模实验可重跑，数据与指标检查通过
B 研究验证	动态商品库、预算路由、消融与失败分析	公平对照结果、多个种子、完整成本记录
C 产品闭环	推荐接口、会话反馈、导入发布与前端	可现场完成浏览、反馈、新品发布和策略对比
D 稳定与优化	任务恢复、版本切换、负载测量、可选 C++	故障恢复记录、正确性对照与性能报告
E 展示交付	固定环境、示例数据、讲解与复现文档	在准备好的环境中独立启动并重放展示流程

个人实现时优先建立 Python、PyTorch 与数据评测能力，再完善服务和展示，最后根据实测决定 C++ 投入。各阶段以证据验收，不把框架安装成功等同于功能完成。

与 PRD 指标的衔接

现有 PRD 的推荐 p95 不超过 300 ms、预热后 20 请求每秒持续 10 分钟，以及新品处理时效等要求，均为规划目标。验收前必须固定硬件、模型、候选规模、数据版本和负载分布。未达到目标时如实记录瓶颈及可用范围，不用本报告的选型理由替代性能测量。

算法指标采用相同协议比较 NDCG、Recall 和分组表现；服务指标记录成功率、延迟、回退与错误；新品能力验证其发布、候选可达与过滤合法性。三类证据共同说明最终系统的完成程度。

选型结论

推荐采用“Python 算法主体、版本化数据与索引、独立业务状态、可交互展示、按需 C++ 优化”的实现路线。它使项目的主要投入集中在推荐问题、实验可信度和真实演示上，并为后续性能与规模演进保留清晰边界。

研究结果可以是不显著提升或某些条件下退化；只要协议清楚、分析充分，依然能够形成有价值的技术报告。最终竞争力来自能够独立解释和验证的工作：基线为何成立、改动影响了什么、新品在哪条路径可达、计算成本如何变化，以及系统如何保持正确运行。

14 参考资料与证据说明

资料查阅日期：2026 年 9 月 14 日。正文中的框架能力依据以下官方文档与原作者实现；架构划分、引入顺序和验收方法为本报告针对 EvoRec 的设计判断。文档链接可能更新，正式实施时还需保存仓库提交、依赖锁文件与模型版本。

- 1. LIGER 官方实现：基线代码、训练与运行说明。
- 2. LIGER 环境文件：复现依赖及版本约束。
- 3. Sentence Transformers 使用说明：编码与查询、文档表示接口。
- 4. Faiss 索引说明：精确检索、近似索引和存储计算。
- 5. DuckDB Parquet 文档：直接读写、列选择和过滤处理。
- 6. FastAPI 功能文档：类型校验、OpenAPI 和接口文档。
- 7. FastAPI 并发与并行说明：异步等待与计算任务的区别。
- 8. PostgreSQL 事务文档：数据库事务语义。
- 9. React 应用构建文档：应用构建工具与工程组成。
- 10. Ant Design 文档：React 组件与 TypeScript 支持。
- 11. Apache ECharts：图表与可视化能力。
- 12. Hydra 官方文档：配置组合、覆盖与多配置运行。
- 13. PyTorch TensorBoard 文档：训练日志写入接口。
- 14. PyTorch 可复现性说明：随机性与跨环境复现边界。
- 15. pybind11 并发与 GIL 说明：Python 与 C++ 的调用边界。
- 16. Docker Compose 文档：服务、网络、数据卷及启动模型。

本地需求依据：《EvoRec 智能商品推荐平台产品需求文档》V1.0，文件 EvoRec_展示版PRD.md。本文不修改原 PRD 的产品范围和验收口径。